Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Genel Konular

- Bu derste, Amaç Hoca ile hazırlanmış bir slayt üzerinden akış diyagramı (flowchart) notasyonu örneklerle tekrar edilir. Ardından C diline giriş yapılır: derleme süreci, IDE, noktalı virgül kuralları, değişkenler, veri tipleri, `printf` / `scanf` ve tip dönüşümleri.
- Geçen dönem nereye kadar gelindi?
  - Cihan Hoca'nın şubesi C'de stringlere kadar gelmiştir. Amaç Hoca'nın şubesi daha çok akış diyagramı üzerinden ilerlemiştir. Fonksiyon ve pointer konuları görülmemiştir.
  - Ders kitabı olarak Kaufman'ın "Problem Solving and Program Design in C" kitabı önerilir. Coursera ve MOOC platformlarındaki dersler de kaynak olarak kullanılabilir.
- Akış diyagramı sembolleri (YTÜ notasyonu)
  - Başla / Bitir: yapısal programlamada başlangıç ve bitişin açıkça gösterilmesi önemlidir. Pascal'daki `begin ... end.` ve C'deki süslü parantezler (`{ }`) aynı fikrin yansımasıdır.
  - Veri girişi (kırık dikdörtgen): içine yalnızca değişken adları yazılır (ör. `A, B`). Girilen değerler sırasıyla bu değişkenlere atanır. Kullanıcıya "sayı giriniz" gibi bir mesaj verilecekse bu, girişten önce ayrı bir çıkış sembolüyle yapılır.
  - Veri çıkışı (alt kenarı dalgalı sembol): değişkenler ve tırnak içinde metinler yazılır (ör. `i, "*", A, "\n"`).
  - İşlem (dikdörtgen): atama ve matematiksel işlemler (ör. `A = 3`, `B = A + 2`). Bir kutuya birden fazla işlem alt alta yazılabilir.
  - Şart (kulakları uzatılmış altıgen): bir T (true) bir F (false) kolu vardır. Kollar aşağıda birleşir ve akış aşağı doğru devam eder.
  - `for` döngüsü: başlangıç değeri, artış miktarı ve bitiş değeri yazılan özel bir kutu ile kapalı bir çevrimden oluşur (ör. `I ← 1`, artış `1`, bitiş `N`).
  - `while` döngüsü: şart altıgeni başta, gövde altında, gövdeden sonra şarta geri dönen bir ok vardır.
  - `do-while` döngüsü: önce gövde, sonra şart gelir. Şart doğruysa gövdenin başına geri dönülür.
  - Bağlantı noktası: kağıtta yer kalmazsa numaralı bir çıkış (ör. 1) konur ve akış başka bir yerde aynı numarayla devam eder.
- `for` sembolünde bitiş değeri
  - Hoca, bitiş değerinin döngüye dahil olmadığını kabul eder: `I ← 1 … N` yazıldıysa döngü `I < N` için çalışır. Bitiş değerinin dahil olması isteniyorsa bunun ayrıca belirtilmesi (ör. eşittir işaretiyle) gerektiğini söyler.
  - Geçen dönem bazı hocaların bitiş değerini dahil ettiğini, bu yüzden bunun yanlış sayılmayacağını ekler.
- Şartlı yapı (`if`) kuralları
  - `if`'in bir kolu yukarı, bir kolu aşağı gitmez. Kollardan biri yukarı gidiyorsa o yapı bir döngüdür, `if` değildir.
  - `if` ile döngü kurulmaz. C'de de `if` ve `goto` ile döngü yazılmaz.
  - False kolunda yapılacak bir şey yoksa kol boş bırakılır ama yine de çizilir. Kodda ise boş `else` yazılmaz.
  - T ve F kolları sağa veya sola yazılabilir. Bunun kesin bir kuralı yoktur, akış boyunca tutarlı olmak da zorunlu değildir. Hoca alışkanlık olarak T'yi sağa yazar.
  - Şartlarda eşitlik için == kullanılması beklenir. Kollar üzerindeki okların çizilmesi zo-

runludur.

- Döngü türlerinin farkı
  - `for` ve `while` döngülerinde şart baştan yanlışsa gövde hiç çalışmaz.
  - `do-while`'da gövde, şart doğru da olsa yanlış da olsa en az bir kez çalışır. Örnek: bir siteye giriş yaparken şifre en az bir kez girilir, yanlışsa tekrar istenir.
  - Döngü şart doğru olduğu sürece döner. "While", "iken" demektir. Şart yanlış olunca döngüden çıkılır.
- Birden fazla şart
  - Şartlar `and` (ve) ile bağlanırsa hepsinin doğru olması, `or` (veya) ile bağlanırsa birinin doğru olması yeterlidir.
  - 3 şart varsa $2^3 = 8$ farklı durum oluşur. Bunlar parantezlerle ve mantıksal bağlaçlarla açıkça yazılmalıdır.
  - Örnek: "devamsızlık < 20 ve ortalama ≥ 40" şartı, "not (devamsızlık ≥ 20 veya ortalama < 40)" ile eşdeğerdir. Bu, De Morgan kuralıdır ve Lojik Devreler ile Ayrık Matematik derslerinde işlenir.
- Programlama dilleri
  - Makine dili ve assembly (1940-50'ler) işlemcinin komut setine bağlıdır.
  - Yüksek seviyeli diller konuşma diline yakındır (ör. `A = B + C - 2`). Bir derleyici (compiler) programı bütün olarak, bir yorumlayıcı (interpreter) satır satır makine diline çevirir. Python ve MATLAB yorumlayıcılı dillerdir.
  - Yüksek seviyeli dillerin avantajları: taşınabilirlik (aynı kod başka makinede derlenip çalışabilir), okunabilirlik ve anlaşılabilirlik, sürdürülebilirlik (değişiklik ve hata ayıklama kolaydır).
  - Dezavantajı: makine diline dönüşümde hız ve bellek açısından bir miktar kayıp olabilir. Derleyiciler bunu optimize etmeye çalışır. Gömülü sistemlerde kritik kısımlar assembly ile yazılarak hızlandırılabilir. Bu da bir trade-off'tur.
  - Standart C kodu taşınabilirdir. Windows'ta yazılan kod Linux'ta da derlenir. Ama işletim sistemine özgü komutlar (ör. `system("cls")` bir DOS komutudur) taşınabilirliği bozar. Bu durumda işletim sistemi tespit edilip ona göre kod çalıştırılır ya da şartlı derleme (conditional compilation) kullanılır.
  - Derleyici önce kodu token'lara ayırır ve sözcük (lexical), sözdizimi ve anlam (semantic) analizleri yapar. Örneğin `A = B + 3`'te `B`'nin 3 ile toplanabilir bir şey olup olmadığı kontrol edilir.
- C dilinin özellikleri
  - Unix ve işletim sistemleri, mikrodenetleyiciler, gömülü sistemler, ağ cihazları (ADSL modem, switch) ve DSP'ler gibi her yerde kullanılır. Java, C#, C++ gibi diller C'den türemiştir.
  - Derste ANSI C (C89/C90) standardı kullanılır. Ödevler Dev-C++ veya CodeBlocks ile yazılmalıdır. Visual Studio önerilmez.
  - Az sayıda anahtar kelimesi (keyword) vardır, bu yüzden öğrenmesi kolaydır.
  - İlkel tiplerden `struct` ve `union` ile bileşik tipler tanımlanabilir. Belleğe doğrudan erişim mümkündür.
  - "Güçlü ama tehlikeli": dizinin dışına erişime izin verir (range checking yoktur), çalışma zamanında tip kontrolü yapmaz, `float`'ı `int`'e atarken hata vermeden gizli dönüşüm yapar. Derleyici hız için her şeye karışmaz. Kontroller programcıya kalır. Java ise dizi dışına erişimde hata verir.
- Yazılım geliştirme süreci
  - Gereksinimler, yapısal tasarım, proje planı, detaylı tasarım (ör. akış diyagramı), kodlama, hata ayıklama (debugging), test, sunum ve bakım.
  - Köprü metaforu: büyük projelerde hatalar baştan tespit edilmelidir, parçalar önceden hazırlanır (modülerlik). Farkı: köprü statiktir, yazılım ise değişim için tasarlanır ve test edilebilir.
  - Modülerlik: sıralayan ve yazdıran tek bir fonksiyon yerine, sıralama ve yazdırma ayrı fonksiyonlarda yazılmalıdır. Aksi hâlde yalnızca birine ihtiyaç duyan kod kopyalanır (duplicated code).
- Derleme ve bağlama (compile and link)
  - Bir program birden fazla kaynak dosyadan oluşabilir. Her kaynak dosya ayrı ayrı derlenir ve bir obje dosyası oluşur.
  - Obje dosyasında kodun bir kısmı makine koduna çevrilmiştir. `printf` gibi önceden derlenmiş fonksiyonlar ise henüz eklenmemiştir.

  - Linker, önceden derlenmiş kodları (kütüphaneler, kendi yazdığımız başka modüller) obje dosyasına bağlar ve çalıştırılabilir dosya (`.exe`) oluşturur. Bu dosya çalıştırılınca belleğe yüklenir ve proses olur.
  - Java'da kaynak kod önce taşınabilir `.class` dosyalarına derlenir, sonra Java Virtual Machine bunları çalıştırır.
- IDE (Integrated Development Environment)
  - Kod yazma, derleme, bağlama, çalıştırma ve hata ayıklamayı tek yerde yapan yazılımdır.
  - Önerilenler: Dev-C++ (slayta göre Windows, Linux), CodeBlocks (slayta göre Windows, macOS), macOS'ta Xcode. Visual Studio Code bir IDE değil, editördür. Ders sonrası not: Slayttaki platform bilgisi büyük olasılıkla eksiktir. CodeBlocks Linux'ta da vardır, Dev-C++ ağırlıklı olarak Windows içindir.
  - Dev-C++'ta dosya kaydedilirken türü C++ değil C seçilmelidir. Derlerken arka planda `gcc` çağrılır (ör. `gcc deneme.c -o deneme.exe -g3`).
- C programının temel yapısı
  - Her C programında bir `main` fonksiyonu ve onun süslü parantezleri olmak zorundadır.
  - `#include <stdio.h>`, standart giriş-çıkış (standard input output) fonksiyonlarını (`printf`, `scanf`) programa dahil eder. "Stüdyo" değildir.
  - String'ler çift tırnakla, karakterler tek tırnakla yazılır. Çift tırnak yerine iki tek tırnak yazılamaz.
- Noktalı virgül kuralı
  - Noktalı virgül bir emir cümlesini (ekrana yaz, sıfır döndür, değişken oluştur, değer ata) bitirir.
  - `#include` satırlarına, `if`, `for` ve `while` başlıklarına noktalı virgül konmaz. Bunlar emir değil, şarttır.
  - `for (...);` veya `while (...);` yazılırsa döngünün gövdesi boş bir ifade olur. Asıl gövde döngüden sonra bir kez çalışır.
  - Tek istisna `do-while`'dır. `do { ... } while (şart);` sonuna noktalı virgül konur, böylece normal bir `while` ile karışmaz.
- Değişkenler ve sabitler
  - Programlamada RAM modeli kullanılır: girdiler ve ara sonuçlar bellekte ayrılan yerlerde (değişkenlerde) saklanır.
  - Bildirim (declaration): `int a;`. Tanım (definition): `int c = 0;` (bildirim + ilk değer atama). Atama (assignment): `a = 5;`. Hoca "declaration + assignment = definition" olarak özetler.
  - Bir değişkenin etiketi (adı), bellekteki adresi, boyutu (size), değeri ve adresini tutabilecek bir pointer tipi vardır.
  - `i = i + 1` bir eşitlik değil, atamadır. Önce sağ taraf hesaplanır (çarpma toplamadan önce), sonuç en son sol taraftaki değişkene atanır.
  - Sabitler `const int b = 0;` ile veya `#define PI 3.14` makrosuyla tanımlanır. `const` değişkenin değeri çalışma zamanında bellekten okunur. `#define` ise derleme öncesinde, `PI` yazılan her yere değerin birebir kopyalanmasıdır. `#define`'da tip ve noktalı virgül yoktur. Makrolar geleneksel olarak büyük harfle yazılır.
- Değişken adlandırma kuralları
  - Harf, rakam ve alt çizgi (_) kullanılabilir. Alt çizgiyle başlanabilir.
  - Rakamla başlanamaz. Türkçe karakter ve `?`, `!` gibi semboller kullanılamaz.
  - C büyük-küçük harfe duyarlıdır (case sensitive): `sum`, `Sum` ve `SUM` farklı değişkenlerdir. Pascal'da böyle değildir.
  - Ayrılmış anahtar kelimeler (`int`, `for`, `else`, `long`, `auto`, `typedef` vb.) değişken adı olamaz. IDE bunları farklı renkte gösterir.
  - Değişken adları üzerinde çalışılan veriyle anlamlı bir ilişki taşımalıdır.
- Veri tipleri
  - `void`: ileride görülecek genel bir tip.
  - Skaler tipler tek bir büyüklük tutar: tam sayılar, kayan noktalı sayılar, karakterler, pointer'lar ve enumeration. `int`, `int *` ve `int **` birbirinden farklı tiplerdir.
  - Birleşik (aggregate) tipler: diziler ve `struct`. Örneğin derste kullanılan C89'da karmaşık sayı tipi yoktur (C99 ile `_Complex` gelmiştir), ama `float` reel ve sanal kısımlar bir `struct`'ta birleştirilerek tanımlanabilir. Bu, iki ayrı değişken tutmaktan daha kullanışlıdır.
  - C'de `string` tipi yoktur. String'ler karakter dizisi olarak kullanılır.

  - Niteleyiciler (qualifier): `short`, `long`, `signed`, `unsigned`. C'de Java veya C#'taki `uint` gibi ayrı bir tip yoktur. `int`'in önüne `unsigned` yazılır.
  - `char` 1 byte'tır: `unsigned char` 0-255, `signed char` -128 ile 127 arasındadır.
  - `short` genellikle 2 byte'tır: -32768 ile 32767, `unsigned short` 0-65535.
  - `int` 4 byte'tır. `float` 4 byte, `double` 8 byte'tır.
  - Tiplerin boyutu derleyiciye ve işletim sistemine göre değişebilir. Bilgisayarınızda farklı bir değer görmeniz yanlış değildir.
- `sizeof` operatörü
  - `sizeof(tip)` bir tipin byte cinsinden boyutunu verir. Hoca Dev-C++'ta (TDM-GCC 4.8.1, 64 bit) şu sonuçları gösterdi: `char` 1, `short` 2, `int` 4, `long` 4, `float` 4, `double` 8, `long double` 16 byte.
  - `sizeof` bir fonksiyon değil, tek operand alan (unary) bir operatördür. Fonksiyon çağrısında programın akışı fonksiyonun bulunduğu yere gider ve geri döner. Operatörde (`+`, `-`, `sizeof`) hiçbir yere gidilmez, işlem bulunulan yerde yapılır.
- Literaller
  - Literal, bir değişkene doğrudan atanan sabit değerdir (ör. `22`). Assembly'deki immediate adresleme moduna benzer.
  - Sonuna eklenen harfler tipi belirtir: `u` unsigned, `l` long, `f` float. `2.0` bir `double`'dır, `2.0f` bir `float`'tur.
  - Tam sayılar sekizlik (başında `0`) veya on altılık (başında `0x`) yazılabilir. Örneğin `0x21` = 33.
  - Kayan noktalı sayılarda virgül değil nokta kullanılır: `4.0`. `4,0` iki ayrı değer gibi algılanır.
  - Üstel gösterim: `3e2` = 300.0, `3e-2` = 0.03.
- Giriş-çıkış fonksiyonları
  - `printf` ekrana yazar, `scanf` klavyeden okur. `puts` bir string yazar, `gets` bir satır okur, `getchar` tek karakter okur.
  - Format belirteçleri (format specifier): `%c` karakter, `%d` tam sayı, `%s` string, `%ld` long, `%u` unsigned, `%f` float, `%lf` double (long float), `%x` on altılık.
  - Belirteçler, virgülden sonraki parametrelerle sırayla eşleşir. Bazı dillerde `{0}`, `{1}` gibi numaralarla eşleştirme yapılır, C'de ise sıra önemlidir.
  - `printf("%u", -5)` ekrana 4294967291 yazar: -5'in bitleri unsigned olarak yorumlanır ($2^{32} - 5$). Bellekteki değer değişmez, yalnızca ekrana yazılırken farklı yorumlanır. Aynı değişken `%d` ile yazılırsa yine -5 görülür.
  - `scanf`'te karakter dizileri hariç bütün değişkenlerin önüne `&` konur, çünkü okunan değer değişkenin bellekteki adresine yazılır. Bunun nedeni pointer konusunda anlatılacaktır.
  - `scanf("%s", ...)` boşluk görünce durur. Örneğin ad soyad girildiğinde yalnızca ad okunur. Satır sonuna kadar okumak için `scanf("%[^\n]", kelime)` kullanılabilir: `^` "değil" anlamındadır, Enter'a kadar olan her şey alınır.
- Gizli (implicit) tip dönüşümü
  - Farklı tipler arasında işlem yapılınca küçük tip büyük tipe genişletilir (integral widening). `2 * 1.2` işleminde 2, `double`'a çevrilir ve sonuç `double` olur.
  - Sonuç `int` bir değişkene atanırsa ondalık kısım atılır: 2.4 yerine 2 saklanır.
  - `2.0 + 1/2` = 2.0'dır, çünkü `1/2` iki tam sayının bölümüdür ve 0 verir. `2.0 + 1.0/2` = 2.5'tir.
  - `-3/4` = 0'dır. -0.75 elde etmek için operandlardan biri ondalıklı yazılmalıdır (`-3.0/4`).
  - C'de büyük bir tipi küçük bir tipe atamak yasak değildir, ama değer kaybı olabilir. Örneğin `int`'e sığmayan büyük bir `unsigned int` değeri `int`'e atanırsa negatif veya farklı bir sayı görülür. Java bu tür atamalarda derleme hatası verir.
- Açık (explicit) tip dönüşümü (casting)
  - `f = (float) j / k;` ile `j` önce `float`'a çevrilir. `j = 3`, `k = 2` için `j / k` 1 verirken `(float) j / k` 1.5 verir.
- Operatörler ve operandlar
  - Unary (tek operand): `sizeof`, `++`, `--`.
  - Binary (iki operand): `+`, `-`, `==`, `!=`, `>=`, `<=` vb. En kalabalık gruptur.
  - Ternary (üç operand): `şart ? a : b`. Şart doğruysa `a`, yanlışsa `b` sonucu üretilir.

Hocanın Özellikle Vurguladığı Kısımlar

- `if`'in kolları her zaman aşağıda birleşir. Bir kolu yukarı giden yapı döngüdür. `if` ile döngü kurulmaz.
- Veri girişi sembolünün içine mesaj değil, değişken adı yazılır. Mesaj ayrı bir çıkış sembolüyle verilir.
- False kolu boş olsa da akış diyagramında çizilmelidir.
- Bir algoritmayı anlamak için analiz edilir: her adımda değişkenlerin değerleri takip edilir. Bu, Devre Teorisi'nde devreyi analiz etmeye benzer.
- `for` ve `if` bloklarında, tek satır olsa bile süslü parantez kullanın. Uzmanlaştıkça bırakabilirsiniz.
- Ödevlerde Dev-C++ veya CodeBlocks kullanın, dosyayı C olarak kaydedin. Visual Studio kullanmayın.
- C her işi programcıya bırakır: dizi sınırlarını, tip uyumunu ve dönüşümleri siz kontrol etmelisiniz.
- Hata mesajlarından korkmayın. Çok hata yaparak öğrenilir.
- Akış diyagramında `i = i + 1` yazılması daha doğrudur. `i += 1` C'ye özgüdür. Hoca buna takılmaz, ama ileriki derslerde takılan hocalar olabilir.

Kısa Tekrar Notları

- Akış diyagramı: başla/bitir, giriş, çıkış, işlem, şart, `for`, `while`, `do-while`, bağlantı noktası.
- `for`/`while` hiç çalışmayabilir, `do-while` en az bir kez çalışır.
- Döngü şart doğru oldukça döner.
- Derleme: kaynak kod, derleyici, obje dosyası, linker, executable, proses.
- Noktalı virgül: emirlerin sonuna. `if`, `for`, `while` başlıklarına değil. İstisna: `do { } while ();`.
- Declaration `int a;`, definition `int a = 0;`, assignment `a = 5;`.
- `const` bellekte saklanır, `#define` derleme öncesi kopyalanır.
- Adlandırma: harf, rakam, `_`. Rakamla başlamaz. Türkçe karakter yok. Case sensitive.
- Format: `%c %d %s %ld %u %f %lf %x`.
- `scanf`'te `&` (diziler hariç).
- `1/2 == 0`, `1.0/2 == 0.5`.
- Cast: `(float) j / k`.
- Ternary: `şart ? a : b`.

Detaylı Açıklamalar

Akış diyagramı örnekleri

Derste slayttaki akış diyagramları öğrencilerle birlikte adım adım çözüldü. Aşağıdaki sonuçlar, ders sonrasında aynı diyagramların C ile yazılıp `gcc -Wall` ile derlenmesiyle de kontrol edildi.

Örnek 1 (`for`): `A = 0`, `N = 5`. Döngü `I ← 1`'den `N`'e (N hariç) 1'er artar. Gövdede `A = A + i` yapılır ve `i, "*", A, "\n"` yazdırılır.

```
1*1
2*3
3*6
4*10
```

Bir öğrenci 5 ile biten bir çıktı söyledi (`5*15`). Hoca bitiş değerini dahil etmediği için çıktının 4 satır olduğunu belirtti.

Örnek 2 (`if`): Şart `A mod B == 0`. Doğruysa `K = A / B` hesaplanır ve `A, "/", B, "=", K` yazdırılır. Yanlışsa `A, "%", B, "!=0"` yazdırılır.

- `A = 10`, `B = 4`: `10 % 4 = 2`, şart yanlış. Çıktı: `10%4!=0`.
- `A = 8`, `B = 4`: şart doğru, `K = 2`. Çıktı: `8/4=2`.

Örnek 3 (`while A != B`): `A = 8`, `B = 2`, `K = 0`. Gövde: `B = B * B`, `K = K + 1`. `K mod 2 == 0` ise `B`, değilse `A` yazdırılır. Sonra `"\n---\n"` yazdırılır.

```
8
---
16
---
8
---
65536
---
...
```

`B` 2, 4, 16, 256, 65536 diye büyür ve hiçbir zaman 8 olmaz. Döngü sonsuza kadar döner.

Örnek 4 (`while A >= B`): Aynı başlangıç değerleri. Bu kez false kolunda yazdırmadan önce `A = A + 1` yapılır.

- 1. tur: `B = 4`, `K = 1` (tek). `A = 9` yapılır ve 9 yazdırılır.
- 2. tur: `9 >= 4`, `B = 16`, `K = 2` (çift). 16 yazdırılır.
- `9 >= 16` yanlış, döngüden çıkılır.

```
9
---
16
---
```

Son değerler: `K = 2`, `A = 9`, `B = 16`.

Örnek 5 (`do-while A > B`): `i = 0`, `B = 1`, `A = 10`. Gövde: `A = A - i`, `i = i + 3`.

Tur	A	i	A > B?
1	10	3	evet
2	7	6	evet
3	1	9	hayır

Son değerler: `i = 9`, `B = 1`, `A = 1`.

Örnek 6 (`do-while A < B`): Aynı başlangıç değerleri. Gövde: `i = i + 2`, `A = A - i`. İlk turda `i = 2`, `A = 8`. `8 < 1` yanlış olduğu için döngüden çıkılır, ama gövde bir kez çalışmıştır. Son değerler: `i = 2`, `B = 1`, `A = 8`.

Örnek 7 (iç içe döngü): `i = 1`, `B = 3`, `A = 7`, `y = 5`, `T = 0`. Dıştaki `do-while A > B` döngüsünün içinde `A = A - i`, `k = 2` yapılır, sonra `while k <= y` döngüsü çalışır. İçteki döngünün gövdesi: `T = T * 2 + i`, `k = k + 3`, `T, ";"` yazdır. İçteki döngüden sonra `i = i + 3` yapılır.

- Dış tur 1: `A = 6`, `k = 2`. İç döngü: `T = 1` (`k = 5`), `T = 3` (`k = 8`). `i = 4`. `6 > 3`, devam.
- Dış tur 2: `A = 2`, `k = 2`. İç döngü: `T = 3 * 2 + 4 = 10` (`k = 5`), `T = 24` (`k = 8`). `i = 7`. `2 > 3` yanlış, çıkış.

Çıktı: `1;3;10;24;`. Son değerler: `i = 7`, `A = 2`, `B = 3`, `T = 24`.

`k = 5` iken `k <= y` (`5 <= 5`) doğrudur, bu yüzden iç döngü bir tur daha çalışır. Bir öğrenci şart sağlanınca döngüden çıkıldığını sanmıştı. Hoca, `while`'ın şart doğru olduğu sürece döndüğünü tekrarladı.

Noktalı virgül denemesi

Hoca Dev-C++'ta şu kodu derleyip çalıştırdı:

```c
int i = 0;
for (i = 0; i < 10; i++);
{
    printf("%d", i);
}
```

`for` satırının sonundaki noktalı virgül döngünün gövdesini boş bir ifade yapar. Döngü hiçbir şey yapmadan 10 kez döner, `i` 10 olunca çıkar. Ardından süslü parantezli blok, döngüden bağımsız olarak bir kez çalışır ve ekrana yalnızca `10` yazılır. Derleyici bu durumda sözdizimi hatası vermez. Herhangi bir şarta bağlı olmayan böyle bloklara anonim blok denir. İçinde tanımlanan değişken yalnızca o blokta geçerlidir. Hoca bu konuyu ileriye bıraktı.

Ders sonrası notlar

- **`for` içinde değişken tanımlama:** Derste "C'de `for` içinde değişken tanımlanmaz" denildi. Bu kural, derste kullanılan C89/C90 standardı için geçerlidir. C99 ve sonrasında `for (int i = 0; i < n; i++)` yazılabilir. Derste C89 istendiği için değişkeni döngüden önce tanımlamak güvenli yoldur.
- **`long` boyutu:** Derste `long`'un genellikle 8 byte olduğu söylendi. Ekrandaki Dev-C++ çıktısında ise `long` 4 byte görünüyor. Windows'ta (64 bit dahil) `long` 4 byte'tır. 64 bit Linux ve macOS'ta 8 byte'tır. Her platformda en az 8 byte olan tip `long long`'dur. Hocanın denediği `sizeof(long long long)` ise geçerli bir tip değildir, GCC bu yüzden "too long for GCC" hatası verir.
- **`printf` ve string birleştirme:** Derste `printf("... %d", a + b)` için "string'in yanına bir şey eklenince her şey string'e dönüşür" ve "operatör önceliği sayesinde önce `a + b` hesaplanır" denildi. Bu açıklama Java'daki `"..." + a + b` davranışına aittir. C'de string ile sayı `+` ile birleştirilemez. `a + b` yalnızca `printf`'e verilen bir argümandır. Argümanlar fonksiyon çağrılmadan önce hesaplanır, `printf` de format string'indeki `%d`'nin yerine bu değeri yazar.
- **`int`'e atamada ondalık kısım:** Derste bu işlem için "yuvarlama" denildi. C'de ondalıklı bir değer `int`'e atanınca yuvarlanmaz, ondalık kısmı atılır (sıfıra doğru kesme). Örneğin 2.9 da 2 olur, -2.9 da -2 olur.
- **`unsigned int` ile `int`:** Bir öğrenci sorusunda `unsigned int` "küçük tip" olarak anıldı. İkisi de 4 byte'tır. Fark yalnızca bitlerin nasıl yorumlandığıdır: `unsigned int` 0 ile $2^{32}-1$, `int` -2^{31} ile $2^{31}-1$ aralığındadır. Bu yüzden iki yönde de dönüşümde değer değişebilir.
- **`gets`:** Derste `gets` bir satır okuma fonksiyonu olarak gösterildi. `gets` dizinin boyutunu kontrol etmediği için güvensizdir ve C11 standardından çıkarılmıştır. Bunun yerine `fgets(kelime, sizeof(kelime), stdin)` kullanılmalıdır.
- **`%[^\n]`:** Derste bu ifade bir düzenli ifade (regular expression) olarak anıldı. C'de buna "scanset" denir. Yalnızca köşeli parantez içindeki karakter kümesini tanır, tam bir düzenli ifade değildir.
- **`scanf`'te `&` ve diziler:** Derste string okurken `&` kullanılsa da olacağı söylendi. Pratikte çoğu derleyicide çalışır, ama `&kelime`'nin tipi `char *` değil, "dizi pointer'ı"dır ve `gcc -Wall` bu satır için uyarı verir. Doğru kullanım `scanf("%s", kelime);` şeklindedir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.